

NPLify · P0 Technical Baseline

Deliverable 1 — Entity-Relationship Diagram & Data Model · Draft v5.2

New XP Technologies Limited

8 October 2026 · Confidential · Internal baseline document

Status and conventions

Status: Draft v5.2 (8 October 2026) carries the three changes v5.0 / v5.1 made to v4.0 — (1) the per-project return receiver group is removed: `sender_return` and `partner_transit` receivers belong to no group, and a return or hop leg inherits its attribution from its parent leg (D13); (2) `RECEIVING_ENTITY_ACCOUNT` carries explicit wallet fields (`account_kind`, `network`, `address`) beside the bank reference (D14); (3) `FACILITATING_ENTITY` is renamed `PARTNER_ENTITY`, and the partner's collection endpoints are merged into one `COLLECTION_RECEIVING_ENDPOINT` table owned by the partner entity (D15; rename pending NPL confirmation) — and adds the four decisions of the review of the v5.1 draft on 8 October 2026: (4) `SETTLEMENT_REGISTRATION` payout details follow the account's kind (D16); (5) `DISBURSEMENT_LINE.deal_id`, required when the paid receiver is group-less (D17); (6) a nullable rail on `COLLECTION_RECEIVING_ENDPOINT`, required before a bank endpoint is active (D18); (7) an endpoint's kind must match the leg's collection method (D19). Everything else carries over unchanged. Supersedes v5.1, v5.0 and v4.0 in full. This baseline is an internal New XP document; the client deliverables are derived from it.

Convention: all monetary amounts are stored as exact decimals; every table carries `created_at` / `created_by`; audit, approvals and locking are modelled once (domain 4) and apply to all master data. Derived figures (client balance, group entitlement, offsets, custody) are **views over postings and deals** — **never stored**.

Six diagrams across four domains, one schema: 54 tables and 4 views. In each diagram, entities defined in another diagram appear with no attributes and name that diagram. Relationship lines run from the referenced row to the row that stores the foreign key; `||--o{` is composition (the parent owns the child), `||..o{` a reference or a polymorphic link resolved by a type column plus an id. Section 6 lists what changed since v4.0; Appendix A reconciles the model with the FRS; Appendix B is the data dictionary.

1 · Configuration domain

A **Project** (= client) owns its configuration. Partners are global counterparties; how a partner is used *in this project* is the **partner configuration** block — fees, vehicles, endpoints, registrations and rebate terms hang off it. Counterparty receivers are organised in **receiver groups**, the unit entitlement is tracked on, and each receiving entity holds one or more **accounts** with a currency and rail.

Diagram 1a. PARTNER_CONFIG, PARTNER_PAIR, SETTLEMENT_REGISTRATION are defined in Diagram 1b.

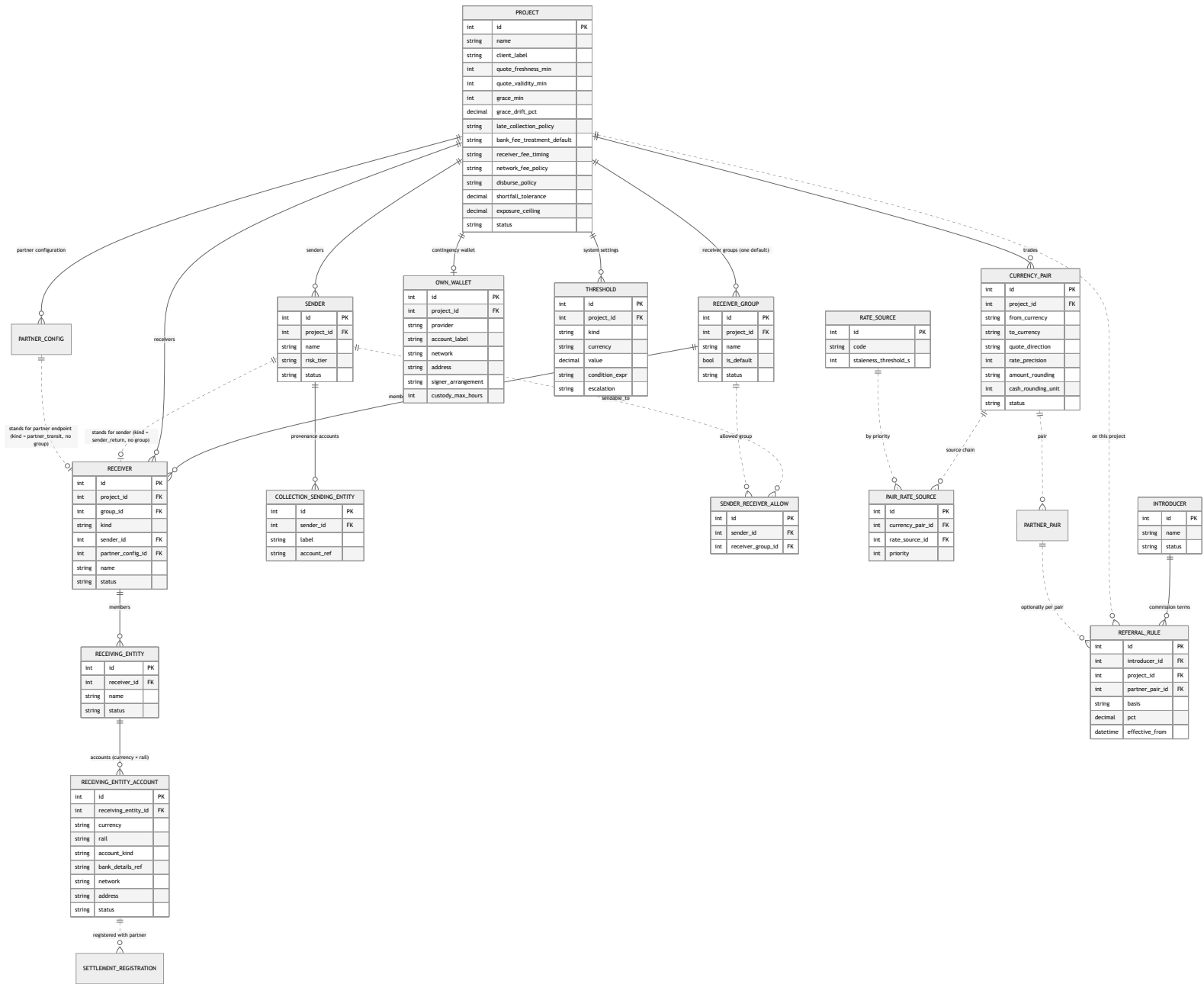


Diagram 1a — Configuration — project, parties and receiver groups

Diagram 1b. PROJECT, CURRENCY_PAIR, SENDER, RECEIVING_ENTITY_ACCOUNT are defined in Diagram 1a.

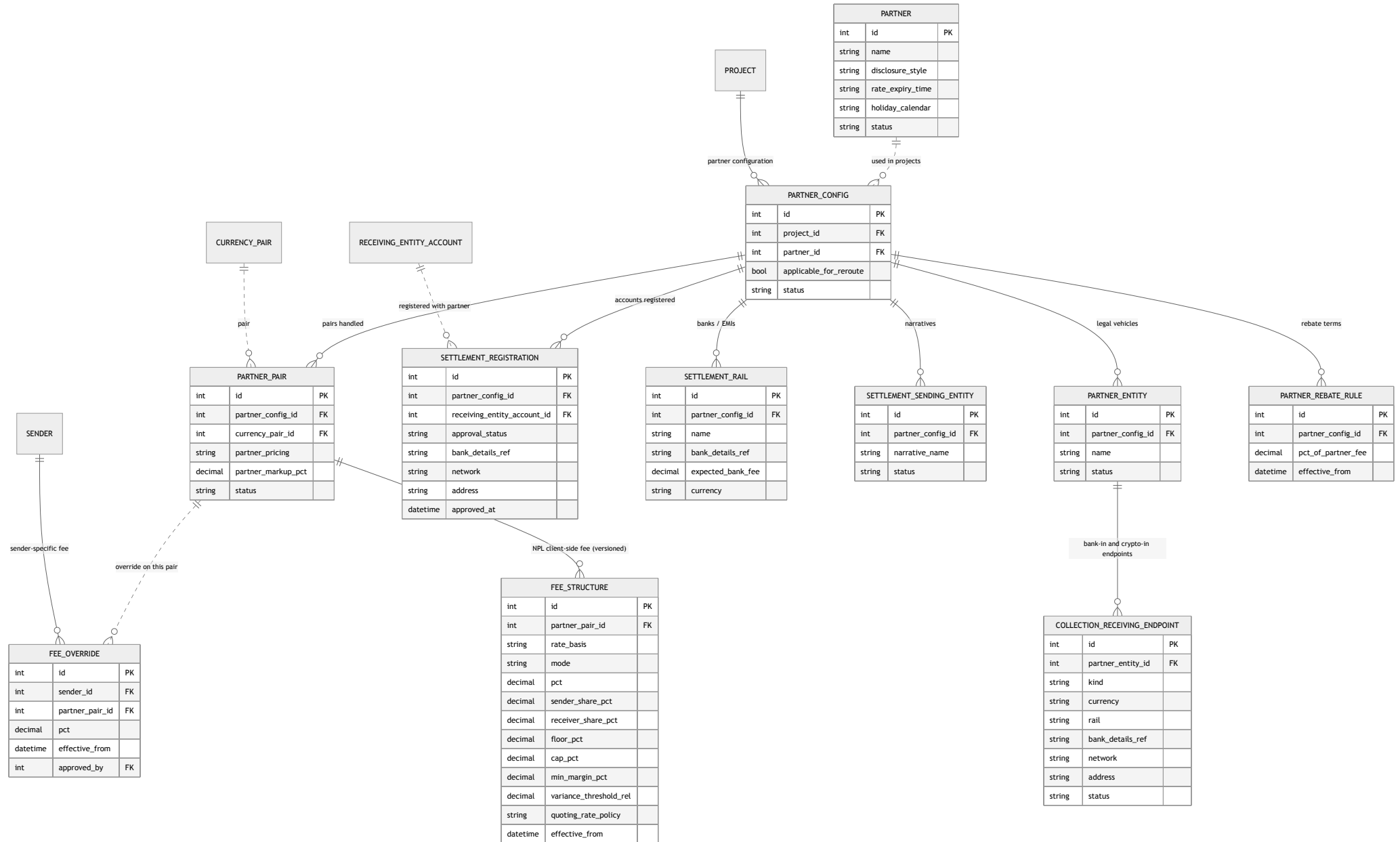


Diagram 1b — Configuration — partner configuration block

Notes

- A **Project** (= client) owns its configuration. Counterparty receivers sit in **receiver groups**: every project has one default group and every counterparty receiver belongs to exactly one group. Entitlement is tracked per group, so payouts within a group are ordinary settlement and only payouts across groups are offsets; a project with a single group never has an offset.
- A **Receiver** has a kind: counterparty for a client receiver; sender_return stands for a sender so refunds use the ordinary payout path and the partner's destination approval; partner_transit stands for a partner's collection endpoint so the hop of a two-leg route is an ordinary payout. sender_return and partner_transit receivers belong to no group (group_id is null). They never appear on forward deals or allow lists — both validate kind = counterparty — and a payout to them is attributed to the group of the leg that pays them, so no entitlement moves between groups on a refund or a hop. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not a group (D13).
- A receiving entity holds one or more **accounts**, each with a currency and rail (EUR via SWIFT and EUR via SEPA are two accounts). An account is a bank account or a wallet (account_kind): a bank account carries bank_details_ref and a bank rail; a wallet carries network and address explicitly, mirroring the partner-side COLLECTION_RECEIVING_ENDPOINT, so addresses can be checksum-validated, checked against the rail, and found when the same address is registered with two partners. Registrations and payout lines point at the account. No table stores actual bank details: bank_details_ref points at the external store.
- Fees are per partner × pair (FEE_STRUCTURE under PARTNER_PAIR), so choosing the partner is part of pricing. PARTNER.disclosure_style records whether the partner's markup is visible; partner_markup_pct is known only for market_plus partners.
- A partner operates through one or more legal vehicles (PARTNER_ENTITY: PT-tour, PT Global Inc); the deal stamps the vehicle at quote so paperwork names the right company. Each vehicle owns the endpoints a sender can pay into (COLLECTION_RECEIVING_ENDPOINT): kind = bank with a currency and bank_details_ref, or kind = wallet with network and address. The partner side therefore mirrors the client side — PARTNER_CONFIG → PARTNER_ENTITY → COLLECTION_RECEIVING_ENDPOINT as RECEIVER → RECEIVING_ENTITY → RECEIVING_ENTITY_ACCOUNT (D15). An endpoint also carries a nullable rail: a bank endpoint must have it before it becomes active, and a wallet may leave it null but must match network when it is set (D18); the endpoint's kind must match the leg's collection_method (D19). Cash-in has no endpoint record; instructions live on the deal. Cash collections round to the nearest 100 / 500 / 1000 units by currency (CURRENCY_PAIR.cash_rounding_unit).
- Destination approval is per partner: SETTLEMENT_REGISTRATION is an account of the project's receiving entity as registered with this partner, carrying the partner's approval status and the payout details the partner holds. Those details follow the account's kind: bank_details_ref for a bank account, network and address for a wallet (D16). SETTLEMENT_SENDING_ENTITY is the narrative on an outgoing transfer; SETTLEMENT_RAIL is the bank/EMI that executes it; neither holds money. COLLECTION_SENDING_ENTITY is provenance only and never a payout destination.
- OWN_WALLET exists for rerouting and pass-through (AQN-LT-Sub is an own wallet at Aquanow); PARTNER_CONFIG.applicable_for_reroute defines the alternate-partner set. INTRODUCER / REFERRAL_RULE record referral commissions NPL pays; PARTNER_REBATE_RULE records rebates a partner pays NPL.

2 · Rates domain

Diagram 2. RATE_SOURCE, CURRENCY_PAIR are defined in Diagram 1a; PARTNER_PAIR in 1b; DEAL, CONVERSION, BALANCE_CONVERSION in 3a.

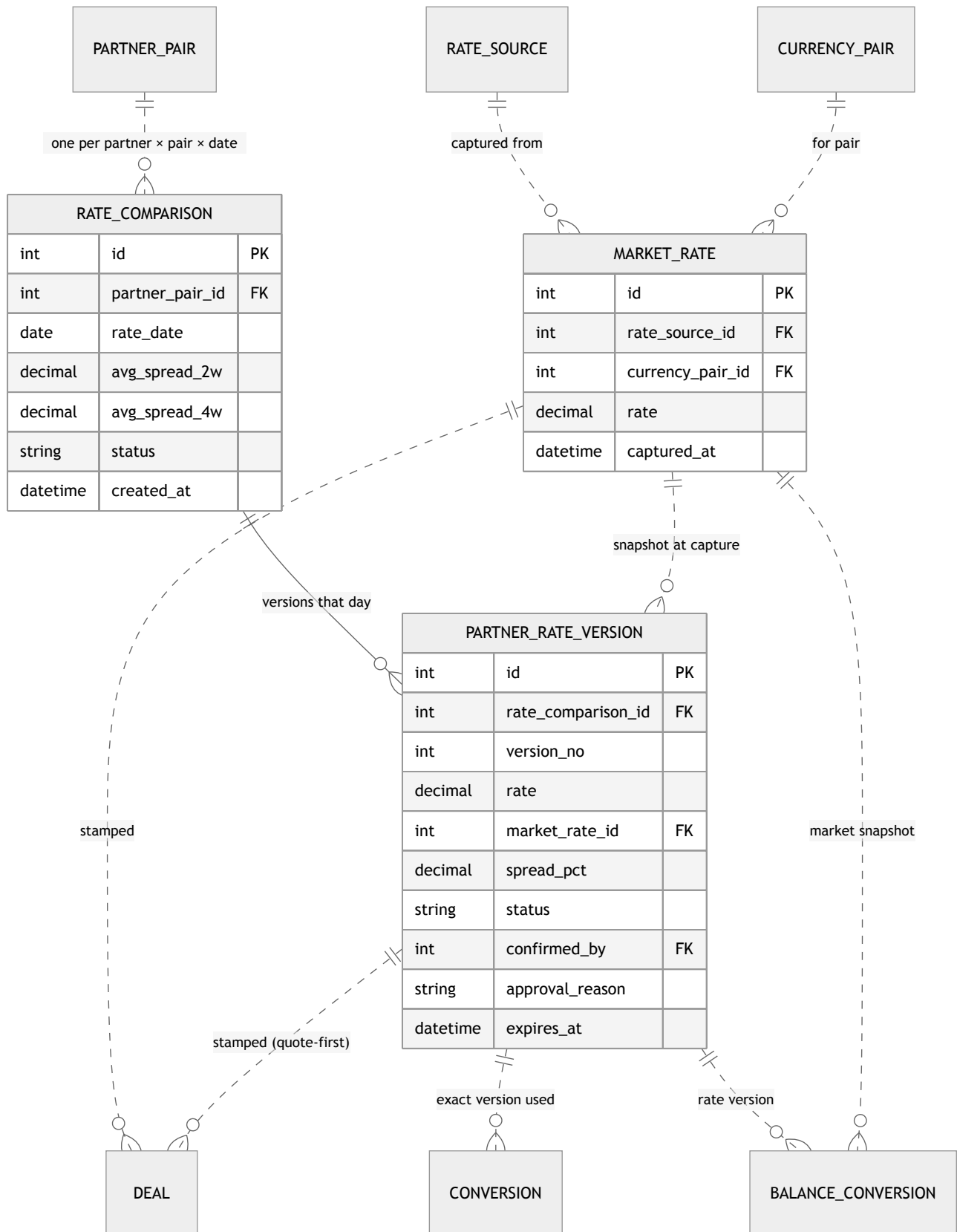


Diagram 2 — Rates

Notes

- The rate comparison for the day is created lazily by the first deal that needs it, one per partner × pair × date, and reused by every deal that day. A partner may give a new rate during the day, so one date can hold several versions. Rolling averages are built from confirmed versions only.
- A version is breached when $\text{spread} > \text{avg_2w} \times (1 + \text{variance_threshold_rel})$; quoting and conversion on that partner × pair freeze until Finance approves with a reason, rejects, or asks for a better rate. Versions are never overwritten; every deal leg, conversion and balance conversion references its exact version.
- Cross-source slippage is a query over MARKET_RATE across sources for the same pair and window.

3 · Deal lifecycle domain

3a · Deal group, legs, collection, conversion, reroute, balance conversion

Diagram 3a. PROJECT, SENDER, RECEIVER_GROUP, OWN_WALLET, COLLECTION_SENDING_ENTITY, RECEIVER are defined in Diagram 1a; PARTNER_PAIR, PARTNER_ENTITY, FEE_STRUCTURE, PARTNER_CONFIG, COLLECTION_RECEIVING_ENDPOINT in 1b; MARKET_RATE, PARTNER_RATE_VERSION in 2; DISBURSEMENT_RETURN in 3b.

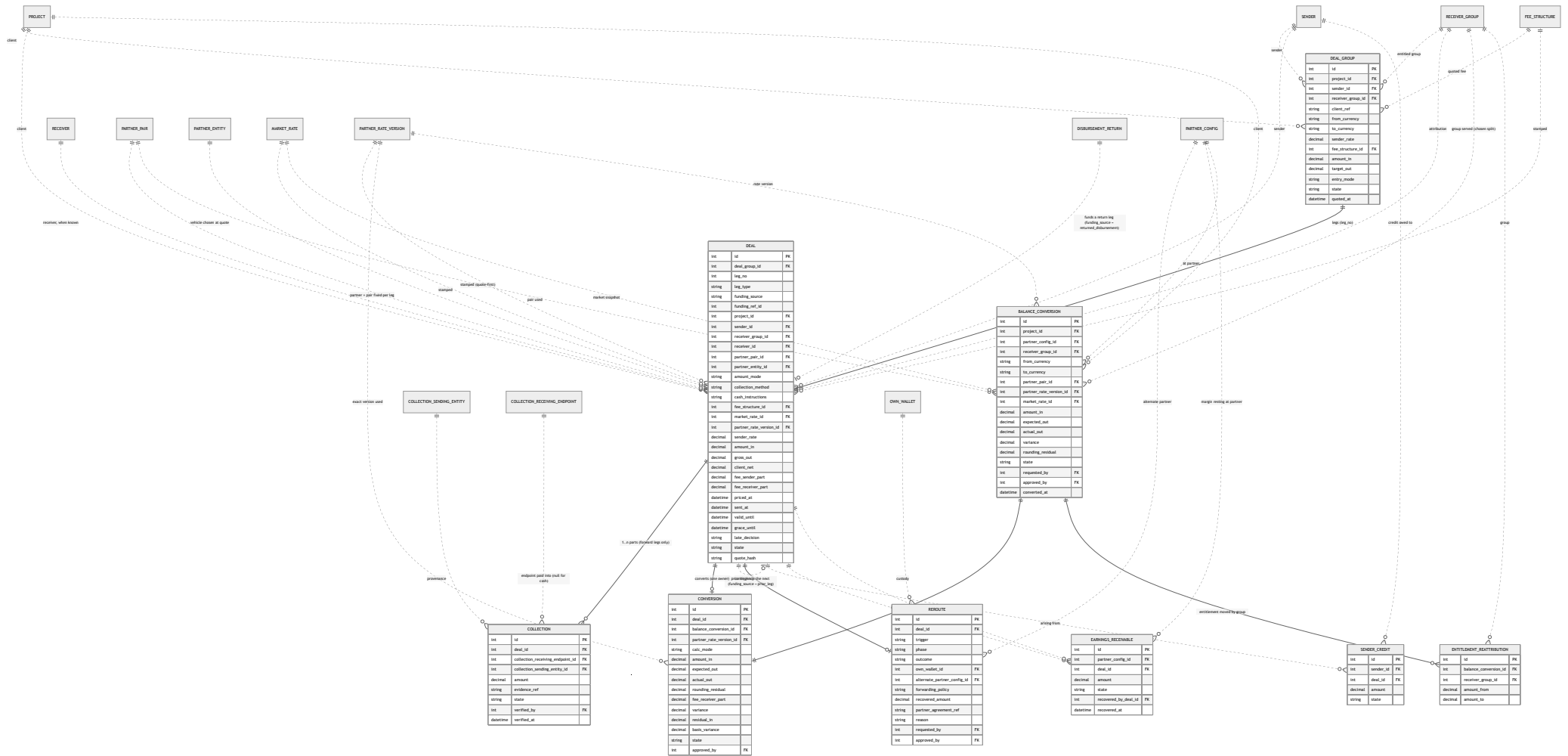
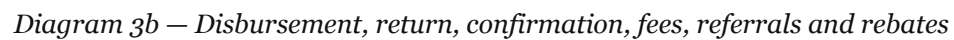


Diagram 3a — Deal lifecycle — deal group, legs, collection, conversion, reroute, balance conversion

3b · Disbursement, return, confirmation, fees, referrals and rebates



Derived views (never stored)

View	Definition
CLIENT_BALANCE(project, partner, currency)	CLIENT_BALANCE(project, partner, currency) = Σ conversions' actual_out booked at that partner (deal legs and balance conversions, target side) – Σ balance conversions' amount_in (source side) + Σ returned amounts – Σ released disbursement totals. Carries rounding dust below one unit.
GROUP_ENTITLEMENT(group, currency)	GROUP_ENTITLEMENT(group, currency) = Σ client_net of converted legs (forward and return) attributed to the group $\pm$ entitlement re-attributions – Σ DISBURSEMENT_LINE.amount on lines whose disbursing deal is attributed to the group + Σ returned amounts on those lines. A payout to a group-less receiver debits the group of the leg that pays it, read from DISBURSEMENT_LINE.deal_id (v5.0 D13; D17 makes the paying leg a stored column). Σ over a project's groups = Σ CLIENT_BALANCE over partners, per currency.
OFFSET(group, currency)	OFFSET(group, currency) = GROUP_ENTITLEMENT; positive = owed, negative = holding another group's flow. Arises only when a counterparty receiver is paid from a leg attributed to another group (v5.0); zero by construction in a single-group project. Carries Management approval and tighter aging.
CUSTODY(own_wallet)	CUSTODY(own_wallet) = Σ collections into the own wallet under open reroutes + Σ recovered dues resting there – Σ forwarded; each row with deal, reason, age. The standing control on contingency holding.

Notes

- **Every deal belongs to a deal group.** DEAL_GROUP is the client-facing instruction: the pair as the sender sees it, the sender rate, fee, reference and entry mode. A plain single-partner deal is a group of one leg; a route that needs two partners (INR → USDT at Ali, USDT → EUR at Jeton) has two forward legs; a rejected payout is unwound with a return leg.
- **Legs.** leg_type forward or return; funding_source collection, prior_leg (leg 2 of a route, no collection) or returned_disbursement (a return leg). Priced once and partner fixed at quote hold per leg. Attribution is to the receiver group: set at quote on the first forward leg, inherited by leg n of a route from leg n–1 and by a return leg from its parent leg. The receiver is optional until settlement.
- Two entry modes as before: collect-first legs are born in state Collected with priced_at = conversion time; quote-first legs carry sent / valid / grace timestamps and a late decision. An estimate-first mode gives an indicative figure and prices at conversion.
- **Conversion has exactly one owner:** a deal leg or a BALANCE_CONVERSION. The receiver-side fee is taken at conversion. Amounts are truncated to whole units at conversion; the dropped fraction (rounding_residual) stays client money in the balance and pays out when it reaches a whole unit. NPL's margin is captured at conversion and is never again at risk from a return or a forward.
- **Balance conversion** is NPL converting a client's held balance (5K SGD → USDT to fund a 15K USDT payout): Finance-initiated, Management-approved, no sender, receiver or collection, same rate machinery, markup by FEE_DECISION. It names the receiver group it serves; ENTITLEMENT_REATTRIBUTION moves that group's entitlement to the new currency, and any excess over the group's entitlement is booked as an approved offset.
- **Reroute** is the only way partner or partner entity change after quote. Forwarding is forward_full or forward_and_recover_dues: NPL nets what the alternate partner owes it (EARNINGS_RECEIVABLE)

against the funds forwarded, within the outstanding receivable, with the partner's agreement recorded. Post-collection outcome is via the own wallet or a refund to the sender.

- One settlement model: DISBURSEMENT drains one partner's holding into one or many DISBURSEMENT_LINES, in whole units. Under disburse_policy to_zero the balance after release is less than one unit; under hold_allowed the balance may rest at the partner within the project's exposure cap.
- A line paying a counterparty receiver whose group differs from the paying leg's attributed group is an offset, created by the same action, with Management approval and tighter aging. A line paying a sender_return or partner_transit receiver is never an offset: those receivers have no group, and the line debits the group of the leg it names in deal_id (D13, D17).
- **Rejected payout.** When the receiving or intermediary bank rejects a transfer, DISBURSEMENT_RETURN records the returned amount, bank fee and reason; the line's outcome becomes returned, a reversing ledger transaction puts the money back into the client's balance and the paying leg's group entitlement, and BANK_FEE_EVENT books the bounce fee. The money then stays in balance for later payouts, or funds a return leg — attributed to the same group — that converts back and pays the sender's return receiver.
- **Fee decisions.** FEE_DECISION is a one-off, approved decision on a single subject: markup applied or waived on a return leg or balance conversion, a network-fee override on a refund, a bank-fee treatment override on one settlement. Standing rules stay in FEE_STRUCTURE and FEE_OVERRIDE.
- CONFIRMATION is full or short; a short confirmation opens a SHORTFALL topped up by a later disbursement. Rounding differences no longer land here. REFERRAL_ACCRUAL and PARTNER_REBATE_ACCRUAL accrue per deal under their rules; rebates are reconciled monthly against the partner's statement.

4 · Ledger & controls domain



Notes

- Client due balances are held as one account per project × partner × currency (CL.<PARTNER>.<CCY>.DUE); group entitlement is a view, not an account. holder_type never includes a settlement rail or narrative entity.
- **Ownership flips client → company at one event only: margin recognition at conversion.** NPL never advances money to a receiver, and withholding on a reroute is not a separate event because the margin is always captured by whichever partner converts.
- Recover dues on a reroute posts own-wallet cash against the earnings receivable at the partner: company money on both sides, no income.
- Transaction sources: Collection, Conversion, BalanceConversion, Disbursement, DisbursementReturn, Confirmation, BankFeeEvent, Reroute, Referral, Rebate, Adjustment. Account purposes include PAYABLE, REFERRAL_PAYABLE and REBATE_RECEIVABLE; ADVANCE does not exist. Cost components include referral and rounding.
- APPROVAL, AUDIT_LOG and RECORD_LOCK are modelled once and apply to all master data. Approver ≠ initiator on every approval; Operations never sees derived economics. Approval actions include refund_to_third_party, cross_group_payout, balance_conversion, recover_dues. cross_group_payout never fires on a line paying a sender_return or partner_transit receiver.

5 · Invariants (enforced at write time)

1. Postings in one transaction sum to zero per currency.
2. Ledger tables are append-only; corrections are reversing transactions.
3. No custody in the normal path: CL.OWN.*.HELD may only be debited by a collection on a deal with an approved REROUTE; the custody aging view is the standing control.
4. Disburse to less than one unit: under disburse_policy = to_zero a DISBURSEMENT pays whole units and leaves only rounding dust; under hold_allowed the balance may rest at the partner within the project's exposure cap (Q5, Q8).
5. Priced once, per leg: a leg's pricing fields are write-once after priced_at; a re-quote is a new leg or a new deal group.
6. Partner fixed at quote, per leg: partner_pair_id / partner_entity_id change only through an approved REROUTE. A collection's endpoint must belong to the leg's partner entity (v5.1, D15). Its kind must match the leg's collection_method — wallet for crypto, bank for bank, no endpoint for cash (v5.2, D19). A COLLECTION may reference only an active endpoint; a bank endpoint is active only with rail set (D18).
7. Entitlement conservation: $\Sigma \text{GROUP_ENTITLEMENT}$ over a project's groups = $\Sigma \text{CLIENT_BALANCE}$ over partners, per currency, including balance-conversion re-attributions. Return and hop legs preserve this because they inherit the parent leg's group (D13). Since v5.2 every payout line resolves to a group — the paid receiver's for a counterparty, the paying leg's via deal_id otherwise — so this is checkable mechanically (D17).
8. Destination approval precedes collection: a quote-first deal cannot leave Inquiry unless at least one account in the entitled group has an approved SETTLEMENT_REGISTRATION with the chosen partner.
9. Approver $\neq$ initiator on every APPROVAL; Operations never receives derived-economics fields.
10. Every balance shown anywhere is a sum over postings or deals.
11. A collect-first deal requires an existing SENDER_RECEIVER_ALLOW entry for its sender and group (Q1).
12. A return leg pays the sender_return RECEIVER linked to the parent leg's sender, unless a Management-approved exception with a reason names another approved receiver.
13. The failed forward line is reversed before a return leg opens, so entitlement and balance are correct at the return conversion.
14. A CONVERSION belongs to exactly one owner: a deal leg or a balance conversion.
15. A recovery on a reroute cannot exceed the outstanding EARNINGS_RECEIVABLE at that partner; the alternate partner's expected conversion is computed on the net amount forwarded.
16. Ownership flips client $\rightarrow$ company at one event only: margin recognition at conversion. NPL never advances money to a receiver (Q13).
17. Offsets arise only when a counterparty receiver is paid from a leg attributed to a different group, and carry Management approval and tighter aging. A project with one group never has an offset; within a group nothing is tracked beyond an informational tally (v5.0).
18. A RECEIVER has a group_id if and only if kind = counterparty. Forward legs and the sender allow list target counterparty groups and receivers only. A return leg inherits receiver_group_id from its parent leg; leg n of a route inherits it from leg n-1 (v5.0, D13).

19. A DISBURSEMENT_LINE paying a group-less receiver (sender_return, partner_transit) carries deal_id; the paying leg's group is a stored fact, never inferred (v5.2, D17).

6 · What changed from v4.0

Three changes came in v5.0 / v5.1 (7 October 2026): the per-project return receiver group is removed (D13), RECEIVING_ENTITY_ACCOUNT gains explicit wallet fields (D14), and the partner's collection side is restructured — FACILITATING_ENTITY becomes PARTNER_ENTITY, and COLLECTION_RECEIVING_ENTITY and WALLET merge into COLLECTION_RECEIVING_ENDPOINT (D15). Two tables became one, so the count is 54 tables and 4 views. Draft v5.2 (8 October 2026) adds the four decisions of the review of the v5.1 draft (D16–D19) and a set of editorial corrections; no table is added or dropped.

D13 — No return group

Decision. sender_return and partner_transit receivers belong to no group. Attribution of a return or hop leg is inherited from its parent leg. Offsets are defined between a leg's attributed group and the group of a counterparty receiver.

Rejected alternative. A dedicated group of kind = return per project holding the sender_return receivers (ERD v4.0 §6; config schema rule "one default and one return group").

Why. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not an entitlement rule, and RECEIVER.kind already enforces it. The return group (a) made every project have at least two groups, so the Understanding document's "a project with a single group never has an offset" was false by construction; (b) classified every refund as a cross-group payout under the v4.0 offset definition, triggering Management approval and offset aging; (c) left the return group's entitlement negative after each refund, breaking invariant 7, unless a reattribution path was added. Removing it restores the Understanding document's §3.3 and §6.3 wording without change. The return group entered during the v4.0 write-up; decision D3 of the review record relied on it and is amended there.

Changed tables

Table	Change
RECEIVER_GROUP	– kind. A group is a group; exactly one is _default per project (D13).
RECEIVER	group_id required when kind = counterparty, null otherwise. sender_return and partner_transit receivers never appear on forward deals or allow lists (validation on kind) (D13).
DEAL	receiver_group_id: set at quote on the first forward leg; a return leg inherits it from its parent leg, leg n of a route from leg n–1 (D13).
DISBURSEMENT_LINE	A line paying a group-less receiver debits the paying leg's group and is never an offset (D13).
RECEIVING_ENTITY_ACCOUNT	+ account_kind (bank / wallet); details_ref → bank_details_ref (bank accounts only); + network, + address (wallets only). Write-time rule: a bank account has bank_details_ref and a bank rail; a wallet has network and address and a crypto rail matching the network (D14).

D14 — Explicit wallet fields on receiving-entity accounts. Rejected: keeping wallet details inside the opaque details_ref. Why: the collection side already stored network and address explicitly; the payout side did not, so addresses could not be validated, checked against the rail, or queried across partners. Bank details stay external by reference, as before.

D15 — Partner entity and collection endpoint

Decision. FACILITATING_ENTITY is renamed PARTNER_ENTITY: the partner's legal vehicle whose name appears on paperwork, the mirror of RECEIVING_ENTITY on the client side. COLLECTION_RECEIVING_ENTITY (bank-in) and WALLET (crypto-in) merge into one COLLECTION_RECEIVING_ENDPOINT table owned by a partner entity, with kind = bank / wallet and the field shape of RECEIVING_ENTITY_ACCOUNT (rail required only on active bank endpoints, see D18). COLLECTION points at one endpoint (collection_receiving_endpoint_id, null for cash) instead of the hand-rolled endpoint_type + endpoint_id pair. The rename is pending NPL confirmation; the FRS term "Facilitating Entity" stays in Appendix A.1.

Rejected alternative. Keeping two endpoint tables, one with a legal owner (wallets under a vehicle) and one without (bank accounts directly under the partner config).

Why. A bank account is always in some legal entity's name, so the v4.0 split left the schema unable to say which vehicle owns a bank-in account, while every wallet knew its owner. One endpoint table under the vehicle fixes the asymmetry, lets the system check that the account a sender paid into belongs to the vehicle stamped on the deal, removes a polymorphic pair from COLLECTION, and makes the partner side read exactly like the client side: config → entity → endpoint.

Table	Change
PARTNER_ENTITY	renamed from FACILITATING_ENTITY; – bank_details_ref (moved to its endpoints).
COLLECTION_RECEIVING_ENDPOINT	new; replaces COLLECTION_RECEIVING_ENTITY and WALLET. partner_entity_id, kind (bank / wallet), currency, bank_details_ref (bank), network + address (wallet), status.
COLLECTION	endpoint_type + endpoint_id → collection_receiving_endpoint_id (null for cash).
DEAL	facilitating_entity_id → partner_entity_id.
Invariant 6	adds: a collection's endpoint must belong to the leg's partner entity.

v5.2 — decisions of the review of the v5.1 draft (8 October 2026)

D16 — Registration payout details follow the account's kind

Decision. SETTLEMENT_REGISTRATION gains network and address (nullable). bank_details_ref ("as paid by this partner") is set only when the registered account's account_kind is bank; for a wallet account the registration carries the network and address as the partner registered them, and the bank field stays empty. The registration remains the partner's view of the destination; RECEIVING_ENTITY_ACCOUNT remains NPL's.

Rejected alternative. Keeping a bank-only field on wallet registrations (the v5.1 shape).

Why. After D14 and D15 every other destination record is kind-aware; a wallet registration had no field that fitted, so an approved wallet destination either carried no details or carried an address in a field named for bank details. Storing the partner's copy also lets the system compare NPL's record with the partner's before paying to a wallet, where a wrong address is unrecoverable.

D17 — DISBURSEMENT_LINE.deal_id

Decision. DISBURSEMENT_LINE gains deal_id, a nullable foreign key to DEAL: required when the paid receiver is group-less (sender_return, partner_transit), optional on counterparty lines. New invariant 19. GROUP_ENTITLEMENT reads the paying leg's group from it.

Rejected alternative. Leaving the rule in prose ("a payout to a group-less receiver debits the paying leg's group").

Why. A DISBURSEMENT pools many deals, and neither it nor its lines named one; the D13 rule and the view that depends on it could not be computed from stored columns, and invariant 7 could not be checked mechanically. The same column will carry the per-line discharge facts of the scenario in which a EUR invoice is paid in USDT in parts.

D18 — Nullable rail on COLLECTION_RECEIVING_ENDPOINT

Decision. The endpoint gains `rail`, nullable. Null means "not yet known", never "does not matter": a bank endpoint cannot become active — and no COLLECTION or payment instruction may reference it — until rail is set; a wallet endpoint may leave rail null (its network already names the road), and when rail is set on a wallet it must match network (mirrors D14).

Rejected alternative. A NOT NULL rail, which forces a guessed value when a partner sends incomplete details; or no rail at all (the v5.1 shape), which made the "same field shape as RECEIVING_ENTITY_ACCOUNT" claim inexact.

Why. The road matters on the collection side as much as on the payout side: the same EUR account reachable by SEPA or SWIFT needs different instructions and arrives with different fees; local rails carry transaction limits (UPI caps, RTGS minimums); and a wrong rail recorded with confidence causes the failures a blank cannot.

D19 — Endpoint kind must match the leg's collection method

Decision. A write-time rule, added to invariant 6: a collection endpoint's kind must match the leg's `collection_method` — wallet for crypto-in, bank for bank-in, and cash-in has no endpoint.

Rejected alternative. None; the rule was missing.

Why. After D15 the fact "how does the money arrive" lives in two places — `DEAL.collection_method` and the endpoint's kind — and nothing stopped them disagreeing silently.

Changed tables (v5.2)

Table	Change
SETTLEMENT_REGISTRATION	+ network, + address (wallet accounts only); <code>bank_details_ref</code> only when the account's kind is bank (D16).
DISBURSEMENT_LINE	+ <code>deal_id</code> — nullable FK to DEAL; required when the paid receiver is group-less, optional on counterparty lines (D17).
COLLECTION_RECEIVING_ENDPOINT	+ <code>rail</code> — nullable; required before a bank endpoint becomes active; on a wallet, must match network when set (D18).
COLLECTION	No column change: may reference only an active endpoint whose kind matches the leg's <code>collection_method</code> (D18, D19).

Views and invariants

Item	Change
GROUP_ENTITLEMENT	Sums <code>client_net</code> over converted legs of both types; a payout line debits the paid receiver's group (counterparty) or the group of the leg named by its <code>deal_id</code> (group-less receivers) — computable from stored columns (D13, D17).
OFFSET	Entitlement — paid per group, arising only when a counterparty receiver is paid from a leg attributed to another group.

Item	Change
Invariant 6	Adds: a collection's endpoint must belong to the leg's partner entity (D15); its kind must match the leg's collection_method (D19); a COLLECTION may reference only an active endpoint, and a bank endpoint is active only with rail set (D18).
Invariant 7	Restated: return and hop legs preserve conservation by inheriting the parent leg's group; since v5.2 every payout line resolves to a group, so the invariant is checkable mechanically (D17).
Invariant 17	Restated: offsets only between a leg's attributed group and a counterparty receiver's group; a single-group project never has an offset.
Invariant 18	group_id iff kind = counterparty; forward legs and the allow list target counterparty groups and receivers only; return and hop legs inherit attribution.
Invariant 19	New: a line paying a group-less receiver carries deal_id; the paying leg's group is a stored fact, never inferred (D17).

Editorial corrections in v5.2

- Invariant 18's note now says the allow list targets counterparty *groups and receivers*, not receivers alone.
- The D13 write-up no longer claims the return group appeared in no decision-record entry: decision D3 of the review record did place sender_return receivers in a return group; the record amends D3 and carries D13–D19.
- The dependent-documents table is refreshed: the data file is already `erd.v5.json` and the decision record carries D13–D19.
- The third v5.1 change (D15) is dated 8 October 2026; this baseline is an internal New XP document from which the client deliverables are derived.

Dependent documents to update

Document	Change
Configuration Schema Specification v1.0	Replace "one default and one return group" with "exactly one default group"; add "every counterparty receiver names a group; no other kind does". Partner section: entities own endpoints; endpoint kind decides which fields are required; rail nullable with the activation rule (D18); registration fields follow the account's kind (D16). Re-validate the FRS Client 1 example — it should pass with one group.
Lifecycle & Approvals v1.0	cross_group_payout must not fire on return legs or hop legs; endpoint activation gated on rail (D18); collection verification checks the endpoint's kind against the leg's collection_method (D19).
Ledger Posting Design v1.0	No account changes. Confirm the return-leg example posts the payout against the parent leg's group entitlement; payout postings may cite the line's deal_id (D17).
Calculation Specification v1.0	No change.
ERD Review — Decision Record	D13–D19 recorded (8 October 2026).
Understanding document v1.1	Nothing to apply; it never had a return group.
data/erd.v5.json, interactive map, knowledge base	Updated to version 5.2 with this draft (fields, views, invariants, notes, vocabulary).

Carried over from v4.0 unchanged

The 12 new tables, 16 changed tables and 3 removals listed in v4.0 §6 (INTRODUCER, RECEIVER_GROUP, RECEIVING_ENTITY_ACCOUNT, REFERRAL_RULE,

PARTNER_REBATE_RULE, DEAL_GROUP, BALANCE_CONVERSION, ENTITLEMENT_REATTRIBUTION, DISBURSEMENT_RETURN, FEE_DECISION, REFERRAL_ACCRUAL, PARTNER_REBATE_ACCRUAL; the removal of RECEIVER_ENTITLEMENT, withhold_earnings and the ADVANCE purpose) all stand. The small gaps noted in the engagement record (EXCEPTION entity, PARTNER_STATEMENT, configuration versioning, SENDER_CREDIT.applied_to_deal_id, DEAL_GROUP.estimate_out, cash rounding per currency, operating time zone) are not addressed in v5.2 and remain open.

7 · Open questions

2 of the questions carried from the Understanding document remain open; the rest were answered by NPL in the review of the Understanding document or settled in the ERD reviews of 6–8 October 2026 and are reflected in this draft.

Still open

#	Question	Blocks	Status
9	FRS figure reconciliation (1,208 vs 1,225; Mode 1 example)	Golden cases	Sud: Needs clarification. To be discussed.
11	Reference workbooks per project archetype	Calculation acceptance	Sud: Will be added soon.

Answered and reflected in v5.2

#	Question	Answer	In the model
1	Collect-first: must SENDER_RECEIVER_ALLOW pre-exist for attribution; handling of inbound matching no sender	Sud: Yes, sender-receiver mapping should be done in configuration. When the transaction is created, the sender is picked from a list.	v4.0: write-time rule that a collect-first deal needs an existing allow entry; the allow list now targets receiver groups.
2	Withholding-eligibility criteria on rerouted deals	Review (7 Oct): NPL's cut is always captured at conversion, so it is never at risk from a return or a forward. If forwarding happens before conversion, whichever partner converts captures it.	v4.0: the eligibility question dissolves. forwarding_policy is forward_full or forward_and_recover_dues; withhold_earnings is removed. Ownership flips at one event only.
3	Post-collection reroute (funds stuck at a partner): transfer, offset, or refund-and-requote	Sud: The partner can send the funds to NPL (and NPL decides where to send) or the partner can refund the sender.	v4.0: REROUTE.outcome via_own_wallet / refund_sender; a refund is a return leg paying the sender_return receiver.
4	Receiving-entity constraints (currency, rail) narrowing settlement choice	Sud: Receiving entities have specific bank accounts with currency and rail restrictions, for example EUR via SWIFT and EUR via SEPA.	v4.0: RECEIVING_ENTITY_ACCOUNT; registrations and payout lines point at the account.
5	Any deliberate partial-disbursement / reserve policy	Sud: Yes, possible. In some projects the client's funds sit until they are ready to receive them, within an exposure threshold NPL sets by appetite.	v4.0: PROJECT.disburse_policy to_zero / hold_allowed with exposure_ceiling as the cap; invariant 4 restated.

#	Question	Answer	In the model
6	Receiver-side fee timing per project	Sud: Receiver-side fee should be applied at the time of conversion, otherwise the balance will be incorrect.	v4.0: default at <code>_conversion</code> ; <code>CONVERSION.fee_receiver_part</code> ; <code>DISBURSEMENT_LINE.receiver_fee_applied</code> becomes exceptional.
7	Network-fee policy on refunds	Sud: Set at project level with per-transaction override (one 100 USD fee was repaid in the next settlement).	v4.0: <code>PROJECT.network_fee_policy</code> plus <code>FEE_DECISION</code> for the override.
8	Whole-unit conversion rounding rule	Review (7 Oct): fractions are dropped to whole units at conversion, per transaction (30.152 → 30). The dropped fraction stays client money, carried in the balance and paid out when it reaches a whole unit. Cash collections round to the nearest 100 / 500 / 1000 by currency.	v4.0: <code>CONVERSION.rounding_residual</code> ; <code>CURRENCY_PAIR.amount_rounding</code> and <code>cash_rounding_unit</code> ; disburse to less than one unit; <code>SHORTFALL</code> no longer takes rounding.
10	AQN-LT-Sub “settled by offset against transactions outside the system”	Sud: This is NPL’s wallet (pass-through at collection or on the settlement side). AQN is Aquanow, a crypto exchange; LT Sub is the account name.	v4.0: <code>OWN_WALLET.provider</code> and <code>account_label</code> ; pass-through is a two-leg route or a reroute via the own wallet.
12	FRS P1 / P3 vs rerouting and multi-partner use — amend FRS or restrict scope	Sud: Check the added explanations (partner types and channels; pass-through and double conversions in §7.1.1). Ask again if still unclear.	v4.0: two-leg routes are deal groups with two legs and a <code>partner_transit_receiver</code> for the hop; FRS principles generalised, scope kept.
13	Advance funding source	Review (7 Oct): never. NPL does not pay a receiver before the client’s money has arrived.	v4.0: <code>LEDGER_ACCOUNT</code> purpose <code>ADVANCE</code> removed; the advance approval and the “advance” ownership-flip event removed.
14	Review point: entitlement split on a balance conversion	Review (7 Oct): chosen group, not proportional. The conversion is triggered by a specific payout.	v4.0: <code>BALANCE_CONVERSION.receiver_group_id</code> ; excess over the group’s source-currency entitlement is booked as an approved offset.

#	Question	Answer	In the model
15	Review point: refunds to a third party at the sender's request	Review (7 Oct): allowed by exception only. Default is the sender's own return receiver; any other destination needs Management approval with a reason and a pre-registered approved receiver.	v4.0: invariant on return legs; APPROVAL action refund_to_third_party.
16	Review point: recover dues on a reroute	Review (7 Oct): yes, NPL nets what a partner owes it against funds about to be forwarded to that partner.	v4.0: forwarding_policy forward_and_recover_dues, recovered_amount, partner_agreement_ref; postings own wallet cash ← earnings receivable, no income.
17	Review point: legs of a route share one client-facing quote	Review (7 Oct): yes, a deal group. The sender saw one price and one reference; margin across legs is the group quote minus the partner conversions.	v4.0: DEAL_GROUP; every deal belongs to a group, a single-partner deal being a group of one leg.
18	Review point (v5.0): does a refund need its own receiver group?	Review (7 Oct, New XP): No. Refund exclusion is a receiver-kind rule; a dedicated return group made every project multi-group (so "a single-group project never has an offset" was false by construction), classified every refund as a cross-group offset needing Management approval, and left its entitlement negative after each refund, breaking invariant 7.	v5.0 (D13): sender_return and partner_transit receivers are group-less; return and hop legs inherit the parent leg's attribution; offsets are defined only against counterparty groups. New invariant 18.
19	Review point (v5.1): restructure the partner collection side — rename FACILITATING_ENTITY, merge the endpoints?	Review (8 Oct, New XP): yes. A bank account is always in some legal entity's name, so the v4.0 split could not say which vehicle owned a bank-in account while every wallet knew its owner. The rename to PARTNER_ENTITY is pending NPL confirmation; the FRS keeps "Facilitating Entity".	D15: FACILITATING_ENTITY → PARTNER_ENTITY (bank_details_ref moved to its endpoints); COLLECTION_RECEIVING_ENTITY and WALLET merge into COLLECTION_RECEIVING_ENDPOINT (kind bank / wallet, same shape as RECEIVING_ENTITY_ACCOUNT); COLLECTION points at one endpoint (null for cash); invariant 6 adds that a collection's endpoint must belong to the leg's partner entity.

Appendix A · FRS reconciliation

A.1 Vocabulary

FRS term	Model entity
Client	PROJECT
Receiving Entity	RECEIVING_ENTITY (member of a counterparty RECEIVER, which belongs to a RECEIVER_GROUP) with its RECEIVING_ENTITY_ACCOUNTs
Sender	SENDER with SENDER_RECEIVER_ALLOW; its refund destination is a group-less sender_return RECEIVER
Partner (PT, Jeton)	PARTNER + PARTNER_CONFIG
Facilitating Entity / Wallet registry	PARTNER_ENTITY with its COLLECTION_RECEIVING_ENDPOINTs (kind = bank / wallet) (D15)
Rate-day versions	RATE_COMPARISON + PARTNER_RATE_VERSION
Settlement	DISBURSEMENT + DISBURSEMENT_LINE + CONFIRMATION (+ DISBURSEMENT_RETURN when rejected)
Shortfall ledger	SHORTFALL + CL.<RECEIVER>.<CCY>.SHORTFALL
PT Comms / H Comms	C0.<PARTNER>.<CCY>.POOL (earnings receivable)
Inter-entity redirection	Offset across receiver groups: a counterparty paid_receiver_id in another group than the paying leg's
Double conversion / pass-through	DEAL_GROUP with two legs (funding_source = prior_leg), hop via a group-less partner_transit RECEIVER

A.2 Deviations from the FRS (classified)

FRS statement	Model	Classification
P1 — no custody	No custody in the normal path; OWN_WALLET + REROUTE for contingency	Decision required (Q12)
P3 — partner books never mix	Accounts per partner never net; alternate partner on reroute; post-collection transfer only if Q3 confirms	Decision required (Q12)
Strict Client → Entity → Sender, one parent each	RECEIVER_GROUP → RECEIVER → RECEIVING_ENTITY → account; SENDER with a group allow-list; FRS project = one default group	Generalisation per NPL instruction
Beyond grace → re-quote always	late_collection_policy per project; FRS project keeps requote	Per NPL instruction (Sept 2026)
Sender always priced on Kraken	FEE_STRUCTURE.rate_basis; FRS project = market	Generalisation
One settlement per deal, full amount	Accumulate → disburse to less than one unit (whole units; rounding dust carried), or hold within the exposure cap where the project allows	Generalisation
Partner fixed at quote	Same, except under REROUTE	Consistent
Bank fee → shortfall → top-up	carry_forward is the FRS behaviour; waived / absorbed added	Per NPL instruction
Registry-only destinations	SETTLEMENT_REGISTRATION.approval_status — FRS project has all approved	Generalisation

FRS statement	Model	Classification
AQN-LT-Sub offset-settled collections	OWN_WALLET at Aquanow; pass-through is a two-leg deal group or a reroute via the own wallet	Resolved (v4.0)

Draft v5.2 — internal baseline. Dependent documents (ledger design, calculation spec, state machines, roles matrix, configuration schema, cutover) to be re-validated against this version per §6.

Appendix B · Data dictionary (field annotations)

Every table also carries `created_at` and `created_by`. Types: int, string, bool, decimal, datetime, date. Rows changed in a revision are marked with the revision and decision that changed them.

B.1 Configuration

PROJECT

Field	Type	Key	Notes
id	int	PK	
name	string		
client_label	string		
quote_freshness_min	int		
quote_validity_min	int		
grace_min	int		
grace_drift_pct	decimal		
late_collection_policy	string		requote / honor_with_approval
bank_fee_treatment_default	string		waived / carry_forward / absorbed
receiver_fee_timing	string		at_conversion (default, Q6) / post_confirmation
network_fee_policy	string		absorbed / charged_back (project default; per-transaction override via FEE_DECISION)
disburse_policy	string		to_zero / hold_allowed (Q5)
shortfall_tolerance	decimal		
exposure_ceiling	decimal		cap on balance resting at a partner
status	string		

PARTNER

Field	Type	Key	Notes
id	int	PK	
name	string		Ali, Jeton, PT
disclosure_style	string		markup_included / market_plus
rate_expiry_time	string		
holiday_calendar	string		
status	string		

RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
code	string		KRAKEN / CMC / OANDA / XE
staleness_threshold_s	int		

INTRODUCER

Field	Type	Key	Notes
id	int	PK	
name	string		

Field	Type	Key	Notes
status	string		

SENDER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		Ayush, Naveen
risk_tier	string		
status	string		

RECEIVER_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		
is_default	bool		exactly one per project
status	string		

RECEIVER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
group_id	int	FK	receiver group; null unless kind = counterparty (v5.0, D13)
kind	string		counterparty / sender_return / partner_transit
sender_id	int	FK	when kind = sender_return
partner_config_id	int	FK	when kind = partner_transit
name	string		the group: Sud, Raj
status	string		

OWN_WALLET

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
provider	string		e.g. Aquanow (Q10)
account_label	string		e.g. LT Sub
network	string		
address	string		
signer_arrangement	string		hardware / multisig
custody_max_hours	int		

THRESHOLD

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	

Field	Type	Key	Notes
kind	string		max_amount / settlement_limit / daily_slots / rate_variation_rel / custom
currency	string		
value	decimal		
condition_expr	string		custom conditions
escalation	string		finance / management

CURRENCY_PAIR

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
from_currency	string		
to_currency	string		
quote_direction	string		out_per_in / in_per_out
rate_precision	int		
amount_rounding	string		truncate_whole_unit (Q8)
cash_rounding_unit	int		100 / 500 / 1000 by currency
status	string		

PARTNER_CONFIG

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_id	int	FK	
applicable_for_reroute	bool		
status	string		

COLLECTION_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
label	string		sender's own account or a client's account
account_ref	string		

SENDER_RECEIVER_ALLOW

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
receiver_group_id	int	FK	sendable_to (was receiver_id)

RECEIVING_ENTITY

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
name	string		NewXP Entity, ReferScout Entity

Field	Type	Key	Notes
status	string		

PAIR_RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
currency_pair_id	int	FK	
rate_source_id	int	FK	
priority	int		

PARTNER_PAIR

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
currency_pair_id	int	FK	
partner_pricing	string		market_plus_pct / disclosed_rate
partner_markup_pct	decimal		known only for market_plus
status	string		

SETTLEMENT_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
narrative_name	string		what the receiver sees
status	string		

SETTLEMENT_RAIL

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		bank or EMI executing the transfer
bank_details_ref	string		
expected_bank_fee	decimal		
currency	string		

FEE_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
partner_pair_id	int	FK	
pct	decimal		
effective_from	datetime		
approved_by	int	FK	

RECEIVING_ENTITY_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
receiving_entity_id	int	FK	
currency	string		
rail	string		SWIFT / SEPA / TRC-20 / ERC-20 / ...
account_kind	string		bank / wallet (v5.0, D14)
bank_details_ref	string		external store; when account_kind = bank (was details_ref)
network	string		when account_kind = wallet
address	string		when account_kind = wallet; checksum-validated per network
status	string		

SETTLEMENT_REGISTRATION

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
receiving_entity_account_id	int	FK	was receiving_entity_id
approval_status	string		pending_partner / approved / suspended
bank_details_ref	string		as paid by this partner; only when the account's kind is bank (v5.2, D16)
network	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)
address	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)
approved_at	datetime		

FEE_STRUCTURE

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_basis	string		market / partner
mode	string		fixed / variable
pct	decimal		
sender_share_pct	decimal		
receiver_share_pct	decimal		
floor_pct	decimal		
cap_pct	decimal		
min_margin_pct	decimal		
variance_threshold_rel	decimal		
quoting_rate_policy	string		worst_applicable / designated / best_applicable
effective_from	datetime		

REFERRAL_RULE

Field	Type	Key	Notes
id	int	PK	

Field	Type	Key	Notes
introducer_id	int	FK	
project_id	int	FK	
partner_pair_id	int	FK	optional
basis	string		share_of_earnings / fixed_pct
pct	decimal		
effective_from	datetime		

PARTNER_REBATE_RULE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
pct_of_partner_fee	decimal		
effective_from	datetime		

PARTNER_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		the partner's legal vehicle: PT-tour, PT Global Inc
status	string		

COLLECTION_RECEIVING_ENDPOINT

Field	Type	Key	Notes
id	int	PK	
partner_entity_id	int	FK	the vehicle whose name is on the account or wallet
kind	string		bank / wallet
currency	string		
rail	string		nullable (v5.2, D18): a bank endpoint needs it to go active; optional for wallets, must match network when set
bank_details_ref	string		external store; when kind = bank
network	string		TRC-20 / ERC-20 / ...; when kind = wallet
address	string		when kind = wallet; checksum-validated per network
status	string		

B.2 Rates**MARKET_RATE**

Field	Type	Key	Notes
id	int	PK	
rate_source_id	int	FK	
currency_pair_id	int	FK	
rate	decimal		
captured_at	datetime		

RATE_COMPARISON

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_date	date		
avg_spread_2w	decimal		
avg_spread_4w	decimal		
status	string		open / closed
created_at	datetime		created lazily by the first deal of the day

PARTNER_RATE_VERSION

Field	Type	Key	Notes
id	int	PK	
rate_comparison_id	int	FK	
version_no	int		
rate	decimal		
market_rate_id	int	FK	
spread_pct	decimal		
status	string		active / usable / breached / approved / rejected
confirmed_by	int	FK	
approval_reason	string		
expires_at	datetime		

B.3 Deal, collection, conversion, reroute, balance conversion**DEAL_GROUP**

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	entitled group
client_ref	string		D-YYMMDD-seq, what the sender sees
from_currency	string		
to_currency	string		
sender_rate	decimal		
fee_structure_id	int	FK	
amount_in	decimal		
target_out	decimal		
entry_mode	string		quote_first / collect_first / estimate_first
state	string		
quoted_at	datetime		

DEAL

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	every deal belongs to a group
leg_no	int		
leg_type	string		forward / return
funding_source	string		collection / returned_disbursement / prior_leg
funding_ref_id	int		DISBURSEMENT_RETURN id or prior DEAL id
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	attribution (D1)
receiver_id	int	FK	optional until settlement
partner_pair_id	int	FK	
partner_entity_id	int	FK	
amount_mode	string		target_out / fixed_in
collection_method	string		crypto / bank / cash
cash_instructions	string		
fee_structure_id	int	FK	
market_rate_id	int	FK	
partner_rate_version_id	int	FK	
sender_rate	decimal		
amount_in	decimal		
gross_out	decimal		
client_net	decimal		
fee_sender_part	decimal		
fee_receiver_part	decimal		
priced_at	datetime		quote or conversion
sent_at	datetime		null on conversion-priced legs
valid_until	datetime		
grace_until	datetime		
late_decision	string		none / honored / requoted
state	string		
quote_hash	string		

COLLECTION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
collection_receiving_endpoint_id	int	FK	the partner endpoint paid into; null for cash (v5.1, D15; was endpoint_type + endpoint_id)
collection_sending_entity_id	int	FK	
amount	decimal		
evidence_ref	string		hash / slip / receipt

Field	Type	Key	Notes
state	string		
verified_by	int	FK	
verified_at	datetime		

CONVERSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	null when owned by a balance conversion
balance_conversion_id	int	FK	null when owned by a deal leg
partner_rate_version_id	int	FK	
calc_mode	string		npl_calculates / partner_calculates
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		booked figure, whole units
rounding_residual	decimal		fraction dropped; stays client money (Q8)
fee_receiver_part	decimal		taken at conversion (Q6)
variance	decimal		
residual_in	decimal		margin pool
basis_variance	decimal		
state	string		
approved_by	int	FK	

REROUTE

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
trigger	string		partner_unavailable / limit_reached / holiday / compliance_hold / other
phase	string		pre_collection / post_collection
outcome	string		via_own_wallet / refund_sender (Q3, post-collection)
own_wallet_id	int	FK	
alternate_partner_config_id	int	FK	
forwarding_policy	string		forward_full / forward_and_recover_dues
recovered_amount	decimal		≤ outstanding receivable at the alternate partner
partner_agreement_ref	string		the partner agreed the net amount
reason	string		
requested_by	int	FK	
approved_by	int	FK	

SENDER_CREDIT

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	

Field	Type	Key	Notes
deal_id	int	FK	
amount	decimal		
state	string		open / applied

EARNINGS_RECEIVABLE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		outstanding / recovered
recovered_by_deal_id	int	FK	
recovered_at	datetime		

BALANCE_CONVERSION

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
receiver_group_id	int	FK	group served; chosen split
from_currency	string		
to_currency	string		
partner_pair_id	int	FK	
partner_rate_version_id	int	FK	
market_rate_id	int	FK	
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		
variance	decimal		
rounding_residual	decimal		
state	string		
requested_by	int	FK	Finance
approved_by	int	FK	Management
converted_at	datetime		

ENTITLEMENT_REATTRIBUTION

Field	Type	Key	Notes
id	int	PK	
balance_conversion_id	int	FK	
receiver_group_id	int	FK	
amount_from	decimal		
amount_to	decimal		

B.4 Disbursement, return, confirmation, fees, referrals, rebates

DISBURSEMENT

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
settlement_sending_entity_id	int	FK	
settlement_rail_id	int	FK	
currency	string		
total	decimal		whole units
bank_fee_treatment	string		
state	string		
prepared_by	int	FK	
approved_by	int	FK	
escalated_to	int	FK	
released_at	datetime		

DISBURSEMENT_LINE

Field	Type	Key	Notes
id	int	PK	
disbursement_id	int	FK	
deal_id	int	FK	nullable; required when the paid receiver is group-less, optional on counterparty lines (v5.2, D17)
settlement_registration_id	int	FK	
paid_receiver_id	int	FK	
amount	decimal		
receiver_fee_applied	decimal		exceptional; fee is normally taken at conversion
reference	string		entity invoice format
outcome	string		sent / confirmed / returned

DISBURSEMENT_RETURN

Field	Type	Key	Notes
id	int	PK	
disbursement_line_id	int	FK	
returned_amount	decimal		
bank_fee	decimal		
reason	string		
returned_at	datetime		

CONFIRMATION

Field	Type	Key	Notes
id	int	PK	
disbursement_line_id	int	FK	
amount_received	decimal		

Field	Type	Key	Notes
slip_ref	string		
result	string		full / short
confirmed_by	int	FK	
confirmed_at	datetime		

SHORTFALL

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
confirmation_id	int	FK	
amount	decimal		
state	string		open / topped_up
topped_up_disbursement_id	int	FK	

BANK_FEE_EVENT

Field	Type	Key	Notes
id	int	PK	
disbursement_id	int	FK	
expected_amount	decimal		
actual_amount	decimal		
treatment	string		waived / carry_forward / absorbed
state	string		open / applied
applied_disbursement_id	int	FK	

FEE_DECISION

Field	Type	Key	Notes
id	int	PK	
subject_type	string		Deal / Disbursement / BalanceConversion
subject_id	int		
fee_kind	string		markup / network / bank
decision	string		apply / waive / override
pct	decimal		
approved_by	int	FK	
reason	string		

REFERRAL_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
referral_rule_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		accrued / paid
paid_at	datetime		

PARTNER_REBATE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
partner_rebate_rule_id	int	FK	
deal_id	int	FK	
expected_amount	decimal		
received_amount	decimal		
period	string		YYYY-MM
state	string		expected / reconciled / received

B.5 Ledger & controls**LEDGER_TRANSACTION**

Field	Type	Key	Notes
id	int	PK	
source_type	string		Collection / Conversion / BalanceConversion / Disbursement / DisbursementReturn / Confirmation / BankFeeEvent / Reroute / Referral / Rebate / Adjustment
source_id	int		
description	string		
posted_at	datetime		
posted_by	int	FK	
reversal_of_id	int	FK	

LEDGER_POSTING

Field	Type	Key	Notes
id	int	PK	
ledger_transaction_id	int	FK	
ledger_account_id	int	FK	
side	string		debit / credit
amount	decimal		
currency	string		
ownership_tag	string		client / company
cost_component	string		principal / earnings / partner_cost / bank_fee / network_fee / variance / referral / rounding

LEDGER_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
code	string		OWNER.HOLDER.CCY.PURPOSE
ownership	string		client / company
holder_type	string		PartnerConfig / OwnWallet / Receiver / Sender / Introducer / Project / None
holder_id	int		
currency	string		
purpose	string		COLLECTED / HELD / DUE / INTRANSIT / PAYABLE / SHORTFALL / CREDIT / POOL / WALLET / EARN_GROSS / EXP_PARTNER / EXP_BANKFEE / EXP_NETWORK / VARIANCE / REFERRAL_PAYABLE / REBATE_RECEIVABLE

Field	Type	Key	Notes
kind	string		asset / liability / income / expense

USER

Field	Type	Key	Notes
id	int	PK	
email	string		Google Workspace
role	string		ops / finance / management
status	string		

APPROVAL

Field	Type	Key	Notes
id	int	PK	
subject_type	string		
subject_id	int		
action	string		
requested_by	int	FK	
approved_by	int	FK	never equal to requested_by
escalated_to	int	FK	
state	string		
reason	string		

AUDIT_LOG

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
field	string		
old_value	string		
new_value	string		
actor_id	int	FK	
at	datetime		

RECORD_LOCK

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
locked_by	int	FK	
locked_at	datetime		